Deuxième partie

Les corrections des travaux dirigés

Correction TD1 :

5.1 Exercice-1

1. ```
   typedef struct {
           int abs;
           int ord;
   }  Point;
   ```
2. ```
   Point SaisirPoint()
   {
           Point p;
           printf("Donner les coordonnees du point :");
           scanf("%d%d",&p.abd,&p.ord);
           return p;
   }
   ```
3. ```
   void AffichePoint(Point p)
   {
        printf("(%d,%d)\n",p.abs,p.ord);
   }
   ```
4. ```
   #define tmax 20


   // Tableau statique
   Point T[tmax];
   int taille, i;

   // contrainte sur la taille du tab statique
   printf("Donner un entier <= %d :",tmax);
   scanf("%d",&taille);


   // Tableau dynamique
   Point *T = NULL;
   // Pas de contrainte sur la taille du tableau dynamique
   printf("Donner un entier :");
   scanf("%d",&taille);
   T = (Point*)malloc(taille * sizeof(Point));
   ```

```
// Dans les 2 situations (statique ou dynamique)
// remplissage du tableau par des points
for(i=0; i < taille ; i++)
    {
        printf("Donner les coordonnees du point:");
        scanf("%d%d", &T[i].abs, &T[i].ord);
    }
```

5. Ecrire une fonction qui recoit en argument un point (une structure du type point) et qui renvoie en résultat la distance du point par rapport à l'origine.
   (a) En transmettant en argument **la valeur** de la structure concernée.

```
float distance(Point p)
{
    return sqrt(p.abs * p.abs + p.ord * p.ord);
}
```

   (b) En transmettant en argument **l'adresse** de la structure concernée.

```
float distance(Point* p)
{
    return sqrt(p->abs * p->abs + p->ord * p->ord);
}
```

6. Donner une fonction qui trie un tableau de points par ordre croissant des distances de ses points (éléments) par rapport à l'origine.

```
void tritab(Point t[], int taille)
{
    int i,j;
    Point aux;
    for(i=0; i<taille-1 ; i++)
        for(j = i+1; j<taille ; j++)
            if (DistTransVal(t[i]) > DistTransVal(t[j]))
                {
                    aux = t[i];
                    t[i] = t[j];
                    t[j] = aux;
                }
}
```

7. Ecrire un programme d'essai des différentes fonctions réalisées.

```
int main(){
        Point a;
        int i, taille;
        a = SaisirPoint();
        AfficherPoint(a);

        /*  Tab Statique et Dynamique*/
        Point TStat[TMAX], *TDyn = NULL;

        /* Lecture de la taille des tableaux */
        printf("Donner la taille (<TMAX) du tableau:");
        scanf("%d",&taille);
```

```
        /* Allocation memoire pour TDyn */
        TDyn = (Point*) malloc(taille * sizeof(Point));

        /* Remplissage du TStat avec des valeurs aleatoire entre et 100 */
        srand(time(NULL)); /* initiliasation du generateur des aleatoires */
        for(i=0; i<taille; i++)
        {
            TStat[i].x = rand() % 100; TStat[i].y = rand() % 100;
        }

        /* Remplissage du TStat avec des valeurs aleatoire entre et 100 */
        for(i=0; i<taille; i++)
        {
            TDyn[i].x = rand() % 100; TDyn[i].y = rand() % 100;
        }

        /* Affichage du TStat */
        for(i=0; i<taille; i++) AfficherPoint(TStat[i]);

        /* Affichage du TDy */
        for(i=0; i<taille; i++) AfficherPoint(TDyn[i]);

        /* tri */
        tritab(TDyn, taille);
        for(i=0; i<taille; i++) AfficherPoint(TDyn[i]);
    return 0;

}
```

5.2 Exercice-2

Définir une structure de données Heure permettant de représenter une heure au format hh/mm/ss, puis écrire les fonctions suivantes :

```
typedef struct
{
    int hh;
    int mm;
    int ss;
}   Heure;
```

1. conversion d'un élément de type Heure en nombre de secondes (entier)

```
int HeureEnSecondes(Heure h)
{
    return (h.hh*3600 + h.mm*60 + h.ss);
}
```

2. conversion d'un nombre de secondes (entier) en un élément de type Heure

```
Heure SecondesEnHeure(int sec)
{
```

```
    Heure h;
    h.hh = sec/3600;
    sec = sec%3600;
    h.mm = sec/60;
    sec = sec%60;
    h.ss = sec;
    return h;
}
```

3. addition de deux éléments de type Heure

```
Heure AddHeures(Heure h1, Heure h2)
{
Heure h3;
return (SecondesEnHeure(HeureEnSecondes(h1)+HeureEnSecondes(h2)));
}
```

4. différence entre deux éléments de type Heure (le temp écoulé entre deux instants).

Correction TD2 :

6.1 Exercice 1

```
#include<stdio.h>
#define N 4
int min(int tab[N])
{
int i;
// 1 premire affectation de index
int index = 0;
// n comparaisons + n-1 affectations
for(i=1; i<N; i++)
{
// n-1 comparaisons
// dans le pire des cas, n-1 affectations
if(tab[i] < tab[index])
index = i;
}
return index;
}
main()
{
    int tt[] ={7,4,2,4,2,9};
    printf("index du min est = %d\n", min(tt));
}
```

- Complexité : T(n) = 1+n+(n-1)+2*(n-1) = 4n-2 ;
- Ordre : O(n) ;

6.2 Exercice 2

```
#define m 3
#define n 3
int MiniMax(int tab[n][m])
{
int i;
```

```
int j;
int max;
int minimax;
// (n) comparaisons
// n -1 affectations
for(i=0; i<n; i++)
{
// n initialisations du max. pour chaque nouvelle ligne
max = tab[i][0];
// (n)*m comparaisons
// (n)(m-1) affectations
for(j=1; j<m; j++)
{
// (n)(m-1) comparaisons
// (n)(m-1) affectations
if(tab[i][j] > max)
max = tab[i][j];
}
// n comparaisons
// 1 affectation
if(i==0)
minimax = max;
// n comparaisons
// (n-1) comparaisons
else if(minimax > max)
minimax = max;
}
return minimax;
}
main(){
    int t[3][3] = {{2,3,4},
    {5,6,7},
    {8,9,10}};
     printf("MiniMax = %d\n", MiniMax(t));
}
```

- Compexité : T(m,n)=4*m*n+3*n-2 ;
- Ordre : O(nm), si m = n alors on est en $O(n^2)$;

6.3 Exercice 3

1. int puissance(a,i)
   int puissance(int a,int i)
   {
   if(i==0)
   return 1;
   else
   return (a*puissance(a,i-1));
   }
2. Complexité de int puissance(a,i). On peut se contenter de la calculer en nombre de multiplications

T(0) = 0 ;
T(1) = 1 + T(0) = 1 ;
T(n) = 1+T(n-1) = 1 + 1 + T(n-2) = n+T(n-n) = n+T(0) = n ;
$T(n) = O(n)$

3. méthode rapide

```
int expRapide(int a, int i)
{
    if (i==0)
            return 1;
  else if (i%2 == 0)
            return expRapide(expRapide(a,i/2),2);
        else return a*expRapide(expRapide(a,i/2),2);

}
```

4. complexité (en nombre de mutilpications) : On évalue la complexité pour les valeurs de n qui sont des puissances de 2 $n = 2^k$.
Pour n=2, P (puissance) = a *a, 1 multiplication (1=ln(2))
Pour n=4, P= $(a * a)^2$, 2 multiplications (2=ln(4))
Pour n=8, P = $((a * a)^2)$, 3 multiplications (3=ln(8))
pour $n = 2^k$, k multiplications (k=ln(n))
On peut déduire que la complexité est $O(ln(n))$

5. somme

```
int somme(int a,    int  n)
{
if(n==0)
return 1;
else
return   (puissance(a,n)+somme(a,n-1));
}
```

6. complexité :
T(s(a,n)) = T(p(a,n))+T(somme(a,n-1)) + 1 (1 addition)
= T(p(a,n))+T(p(a,n-1))+T(somme(a,n-2)) + 1 +1 (2 additions)
= T(p(a,n))+T(p(a,n-1))+T(p(a,n-2))+T(p(a,n-3))+...+T(p(a,1))+ T(s(a,n-n)) + 1+1+1 .....+1 (n additions)
= n + (n-1) +.....+1 + n = n(n+1)/2 + n

est en $O(n^2)$

6.4 Exercice 4

```
int rec1(int n)
{
if(n == 0)
return 1;
else
return 2*rec1(n-1);
}
```

```
int rec2(int n)
{
if(n == 0)
return 1;
else
return (rec2(n-1) + rec2(n-1));
}
```

6.4.1 Le rôle de la fonction rec1(n)

rec1(0)=1 = 2^0
rec1(1) = 2*rec1(0) = 2 = 2^1
rec1(2) = 2*rec1(1) = 2*2 = 4 = = 2^2
rec1(3) = 2*rec1(2) = 2*4 = 8 = 2^3

.
rec1(n) = 2^n
Elle calcule 2^n

6.4.2 Le rôle de la fonction rec2(n)

rec2(0) = 1 = 2^0
rec2(1) = rec2(0) + rec2(0) = 1 + 1 = 2 = 2^1
rec2(2) = rec2(1) + rec2(1)= 2 + 2 = 2^2
rec2(3) = rec2(2) + rec2(2) = 4 + 4 = 2^3

.
rec2(n) = 2^n
Elle calcule 2^n

6.4.3 Complexité de rec1(n)

T1(0) = 0 ; T1(1)=1+T1(0) = 1 ;
T1(n)=1+T1(n-1)=1+1+T1(n-2)=n
La complexité est O(n)

6.4.4 Complexité de rec2(n)

T2(0) = 0 ; T2(1)=1+2*T2(0)=1 ;
T2(2) = 1+2*T2(1) = 1 +2
T2(3)= 1+ 2 T2(2) = 1 + 2 + 2^2
T2(n) = 1+ 2 + 2^2+..... +2^{n-1}
La somme des n premiers termes d'une suite géométrique de raison 2 : $T2(n) = (1 - 2^n)/(1 - 2) = 2^n - 1$
La complexité est $O(2^n)$

Correction TD3 :

7.1 Exercice 1 (Création d'une liste) :

```
/*  Definition de la structure d'un element de la lsite */
struct Element{
int val;
struct Element *suivant;
};

typedef struct Element LISTE;

int InsererElementEnTete(LISTE **L, int valeur)
{
    LISTE * element = NULL;
    element = (LISTE*) malloc(sizeof(LISTE));
    if (element == NULL){
        printf("Probleme d'allocation memoire\n");
        return 0;
    }
    element->val = valeur;
    element->suivant = *L;
    *L = element;
    return 1;
}
```

La complexité : O(1)

7.2 Exercice 2 (Recherche de valeur) :

```
int RechercherValeur(LISTE *L, int valeur)
{
    LISTE *ptr = L;
    while (ptr){
        if (ptr->val == valeur) return 1;
        ptr = ptr->suivant;
    }
```

```
    return 0;
}
```

Complexité : O(n)

7.3 Exercice 3 (Suppression d'un élément) :

```
int SuppressionValeurMin(LISTE **L)
{
    LISTE * ptr = *L, *pMin=NULL, *pPrec = NULL;
    int minim;
    if (!ptr) /* liste vide */
        return 0;
    else if (!ptr->suivant) /* taille de liste = 1*/
        {
            *L = NULL;
            free(ptr);
            return 1;
        }

    minim = ptr->val; pMin = NULL;
    while (ptr->suivant)/*chercher le min et sa position ds la liste*/
    {
        if (minim > ptr->suivant->val){
            minim = ptr->suivant->val;
            pMin = ptr;
        }
        ptr = ptr->suivant;
    }
    if (!pMin){
        ptr = *L;
        *L = (*L)->suivant;
        free(ptr);
        return 1;
    } else{
        ptr = pMin->suivant;
        pMin->suivant = pMin->suivant->suivant;
        free(ptr);
        return 1;
    }
}
```

7.4 Exercice 4(Fusion de deux listes) :

Solution 1 :

```
LISTE *     FusionDe2ListesAlternance(LISTE *L1, LISTE * L2)
{
```

```
    LISTE * ptr1 = L1, *ptr2 = L2, *res = NULL;
    if (L1 == NULL) return L2;
     else if (L2 == NULL) return L1;
    while(ptr1 && ptr2)
    {
        InsererElementEnFin(&res,ptr1->val);
        InsererElementEnFin(&res, ptr2->val);
        ptr1 = ptr1->suivant;
        ptr2 = ptr2->suivant;
    }

    while (ptr1)
    {
        InsererElementEnFin(&res, ptr1->val);
        ptr1 = ptr1->suivant;
    }
    while (ptr2)
    {
        InsererElementEnFin(&res, ptr2->val);
        ptr2 = ptr2->suivant;
    }
    return res;
}
```

Solution 2 :

```
LISTE * Fusion(LISTE *L1, LISTE * L2)
{
   LISTE *p1 = L1, *p2 = L2, *p3, *p4;
   if (!L1) return L2;
   if (!L2) return L1;

   LISTE * L = L1;
   while (p1->suivant && p2->suivant)
   {
     p3 = p1->suivant;
     p1->suivant = p2;
     p4 = p2->suivant;
     p2->suivant = p3;
     p1 = p3;
     p2 = p4;
   }

 if (p1->suivant == 0)  p1->suivant = p2;
 if (p2->suivant == 0) p2->suivant = p3;
 return L;
 }
```

7.5 Exercice 5 (Destruction d'une liste) :

```
void DestructionListe(LISTE **L)
```

```
{
    LISTE *ptr;
    if (*L == NULL) exit(0);
    while (*L != NULL){
        ptr   = *L;
        *L = (*L)->suivant;
        free(ptr);
    }
}
```

7.6 Exercice 6 (Piles) :

```
#include<stdio.h>
#include<stdlib.h>


typedef struct stack {
        int data;
        struct stack *next;
    } STACK;

/*   Empiler   */

void push(STACK **head, int value){
/* create a new node */
STACK *node = NULL;
node = (STACK*)malloc(sizeof(STACK));
if (node == NULL)
{
fputs("Error: no space available for node\n", stderr);
abort();
} else {
/* initialisation */
node->data = value;
/*   insertion*/
if (*head==NULL)
node->next = NULL;
else node->next = *head;
*head = node;
}
}

/* Depiler */

int pop(STACK **head)
{
int value;
if (*head==NULL) {/* pile est vide */
fputs("Error: stack underflow\n", stderr);
```

```
abort();
} else {/* depiler un element */
STACK *top = *head;
value = top->data;
*head = top->next;
free(top);
return value;
}
}


STACK * PairImpair(STACK * P1)
{
STACK * P2 = NULL, *P3 = NULL;
int val;
while (P1)
{
val = pop(&P1);
if (val % 2 == 0)   push(&P2,val);
else push(&P3, val);
}

while (P3)
{
val = pop(&P3);
push(&P2,val);
}

return P2;
}

void AffichePile(STACK *P)
{
while (P)
{
printf("%d\t",P->data);
P = P->next;
}
printf("\n");
}

main()
{
int i;
STACK * P1 = NULL, *P2 = NULL;
for (i=1; i< 10; i++) push(&P1,i);

AffichePile(P1);
P2 = PairImpair(P1);
AffichePile(P2);

}
```

7.7 Exercice 7 (Files) :

```
/* structure de la pile */

typedef struct stack {
        char data;
        struct stack *next;
    } STACK;


/*  Empiler un caractere */

void push(STACK **head, char value){
/* create a new node */
STACK *node = NULL;
node = (STACK*)malloc(sizeof(STACK));
if (node == NULL)
{
fputs("Error: no space available for node\n", stderr);
abort();
} else {
/* initialisation */
node->data = value;
/*  insertion*/
if (*head==NULL)
node->next = NULL;
else node->next = *head;
*head = node;
}
}

/* Depiler un caractere      */

int pop(STACK **head)
{
char value;
if (*head==NULL) {/* pile est vide */
fputs("Error: stack underflow\n", stderr);
abort();
} else {/* depiler un element */
STACK *top = *head;
value = top->data;
*head = top->next;
free(top);
return value;
}
}
```

```
/* structure d'un element (noeud) de la file */
struct queue_node
{
struct queue_node *next;
char data;
};

/* structure de la file */

struct queue
{
struct queue_node *first;
struct queue_node *last;
};


/* Enfiler (rajouter) un element */

int enqueue(struct queue *q, const char value)
{
struct queue_node * node = NULL;
node = (struct queue_node *)malloc(sizeof(struct queue_node));
if (node == NULL) {
printf("probleme dallocation memoire");
return 1;
}
node->data = value;
if (q->first == NULL) {
q->first = q->last = node;
} else {
q->last->next = node;
q->last = node;
}
node->next = NULL;
return 0;
}

/* defiler (supprimer) un elment */

int dequeue(struct queue *q, char *value)
{
if (!q->first) {/* test si la file vide */
*value = 0;
return 1;
}
/* recuperer la valeur du noeud supprimer */
*value = q->first->data;
/* recuperer ladresse du noeud supprimer */
struct queue_node *tmp = q->first;
if (q->first == q->last) {
q->first = q->last = NULL;
```

```
} else {
q->first = q->first->next;
}
free(tmp); /* liberation de lesapce occupe par le noeud suprime*/
return 0;
}

/*  la fonctin EcrireMessage */

struct queue * EcrireMessage(char chaine[])
{
int i,taille = strlen(chaine);
struct queue * q=NULL;
q = (struct queue *)malloc(sizeof(struct queue));
if (!q) { printf("Prob allocation mem "); exit(0);}
q->first = q->last = NULL;
for(i=0;i<taille;i++)
enqueue(q, chaine[i]);
return q;
}

/*    Palindrome ? */

int EstPalindrome(char chaine[])
{
int t1 = strlen(chaine) % 2;
int t2 = strlen(chaine)/2;
int i;
char val, c1, c2;
STACK *p = NULL;
struct queue * Q;
Q = EcrireMessage(chaine);

for(i=0;i<t2; i++)
{
dequeue(Q,&val);
push(&p, val);
}

if (t1 != 0) dequeue(Q,&val);

for(i=0;i<t2;i++)
{
dequeue(Q,&c1);
c2 = pop(&p);
if (c1 != c2) return 0;
}
return 1;
}
```

Correction TD4 :

8.1 Exercice 1 :

```
/*   structure arbre */
typedef int element_type;
typedef struct tree_node *tree_ptr;
struct tree_node {
element_type element;
tree_ptr left;
tree_ptr right;
};
typedef  tree_ptr  SEARCH_TREE;
```

1. Insèrer une valeur dans un ABR.

```
tree_ptr insert( element_type x, SEARCH_TREE T )
{
if( T == NULL )
{   /* Creer et retourner un arbre compose d'un seul noeud*/
T = (SEARCH_TREE) malloc ( sizeof (struct tree_node) );
if( T == NULL )  {printf("Espace memoire insuffisant!!!");
          exit(0);
          }
   else
{
T->element = x;
T->left  =  T->right  =  NULL;
}
}
else    if( x <= T->element )
       T->left  =  insert( x, T->left );
else     if( x > T->element )
T->right = insert( x, T->right );
/* Sinon x existe deja dans l'arbre. On va rien faire */
return T; /* Ne pas oublier cette ligne!! */
}
```

2. Chercher si une valeur donnée appartient à un ABR.

```
tree_ptr   RechercherElement(SEARCH_TREE   Arbre,   element_type   e)
```

```
{
if (Arbre == NULL) return NULL;
if (e == Arbre->element)
return Arbre;
else if (e > Arbre->element)
return RechercherElement(Arbre->right, e);
else return RechercherElement(Arbre->left,e);
}
```

3. Compter le nombre d'occurrences d'une valeur X donnée dans un ABR A.

```
void NombreOccurences(SEARCH_TREE A, element_type e, int *nombre)
{
    if (A != NULL) {
        NombreOccurences(A->left,e, nombre);
        if (A->element == e) (*nombre)++;
        NombreOccurences(A->right,e, nombre);
    }
}
```

4. Afficher les clés de l'ABR en utilisant un parcours infixé.

```
void ParcoursInfixeArbre(tree_ptr Arbre)
{
    if (Arbre != NULL) {
        ParcoursInfixeArbre(Arbre->left);

        /* traitement du cle */
        printf("%d\t", Arbre->element);

        ParcoursInfixeArbre(Arbre->right);
    }
}
```

5. Construire une liste triée à partir d'un ABR.

```
void ConstListeTriee(SEARCH_TREE A, LISTE **L)
{
    if (A != NULL) {
        ConstListeTriee(A->left, L);
        InsererElementEnFin(L, A->element);
        ConstListeTriee(A->right, L);
    }
}
```

6. Construire un ABR à partir d'une liste (non triée).

```
SEARCH_TREE ArbreAPartirDeListe(LISTE *L)
{

    SEARCH_TREE A = NULL;
    while (L != 0)
    {
        A = insert(L->val, A);
        L = L->suivant;
    }
 return A;
}
```

7. Déduire une fonction de tri de liste.

```
LISTE * TrierListe(LISTE * L)
{
    SEARCH_TREE A = NULL;
    LISTE * ptr = NULL;
    A = ArbreAPartirDeListe(L);
    /* Copier les elements de A dans la liste ptr */
    ConstListeTriee(A, &ptr);
    return ptr;
}
```

Programme faisant appel aux différentes fonctions définies ci-dessus :

```
main()
{
    SEARCH_TREE A=NULL;
    tree_ptr ptr = NULL;
    LISTE *L=NULL, *autreL = L;
    int i, nbre;

    /* creation d'un arbre de 10 elements */
    for(i=1;i<=10;i++)
        A = insert(rand()%10, A);

    /* Affichage de l'arbre A */
    printf("\n L'arbre est (infixe) : \n");
    ParcoursInfixeArbre(A);

    printf("\n L'arbre est (prefixe) : \n");
    ParcoursPrefixeArbre(A);

    printf("\n Donner un entier qu on cherche s il existe dans l'arbre :");
    scanf("%d",&i);
    ptr = RechercherElement(A, i);
    if (ptr)
        printf("\n La valeur : %d figure dans l'arbre, son adresse est : %p\n", i, ptr);
    else printf("\n La valeur : %d ne figure pas dans l'arbre A\n", i);


    /* Nombre d'occurence de i*/
    nbre = 0;
    NombreOccurences(A,i, &nbre);
    printf("\n Le nombre d'occurences de : %d est : %d\n", i, nbre);

    /* construction liste triee a partir arbre */
    L = NULL;
    ConstListeTriee(A, &L);
    printf("\n La liste construite est :\n");
    AfficheListe(L);

    /* creer liste non triee */
    for(i=1; i<10;i++)
```

```
    InsererElementEnTete(& autreL, rand()%10);

    printf("\n La liste construite est (non triee) :\n");
    AfficheListe(autreL);

    /* trier la liste */
    autreL = TrierListe(autreL);
    printf("\n La liste construite apres tri :\n");
    AfficheListe(autreL);

    /* ABR, trie ou pas -------------- Exercice 2 */
    if (EstCeQueABR(A)) printf("\n Arbre est ABR.\n");
      else printf("\n Arbre n'est pas un ABR.\n");

/* valeur minimale de l'arbre ---------------------- Exercice 3 */
printf("La valeur min de l'arbre est : %d\n", LePlusPetit(A));

}
```

8.2 Exercice 2 :

Écrire une fonction booléenne (retourne 1 (TRUE) ou 0 (FALSE)) qui teste si un arbre binaire est un ABR. Quelle est sa complexité ?

```
int EstCeQueABR(SEARCH_TREE A)
{
    LISTE * L = NULL;
    ConstListeTriee(A, &L);
    if (A == NULL) return 0;
    else if ((A->left == NULL)&& (A->right == NULL)) return 1;
    else {
        while (L->suivant != NULL)
        {
            if (L->val > L->suivant->val) return -1;
            L = L->suivant;
        }
    }
    return 1;

}

element_type LePlusPetit(SEARCH_TREE A)
{
while (A->left != NULL)
A = A->left;
return A->element;
}
```